AI-BASED USER AND ENTITY BEHAVIOUR ANALYTICS SYSTEM

AN IN-HOUSE PROJECT REPORT

Submitted by

AMAAN SAIFY (2023BCSE07AED025)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Under the Supervision of

Dr. Rathnakar Achary

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

OCTOBER - 2026

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

-

CERTIFICATE

This is to certify that the In-house Project work entitled "AI-BASED USER AND ENTITY BEHAVIOUR ANALYTICS SYSTEM" is the bonafide work done by **AMAAN SAIFY (2023BCSE07AED025)** submitted in partial fulfilment of the requirements for the award of the degree Bachelor of Technology in **Computer Science and Engineering (Cyber Security)** during the year 2026-27.

Dr. Rathnakar Achary
Associate Professor
Dept. of Computer Science and Engineering

DECLARATION

This is to declare that the report titled “AI-Based User and Entity Behaviour Analytics System” has been made for the partial fulfilment of the Programme - Bachelor of Technology in **Computer Science and Engineering (Cyber Security)**, under the guidance of **Dr. Rathnakar Achary**. I confirm that this report truly represents my work undertaken as a part of my **In-house Project** work. This work is not a replication of work done previously by any other person. I also confirm that the contents of the report and the views contained therein have been discussed and deliberated with the Mentor.

NAME	REGESTRATION NO.	SIGNATURE
AMAAN SAIFY	2023BCSE07AED025	

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people that made it possible, whose constant guidance and encouragement crown all the efforts successfully.

I am very thankful to my supervisor **Dr. Rathnakar Achary** for his sustained inspiring guidance and cooperation throughout the process of this project. His wise counsel and valuable suggestions are invaluable.

I would like to thank **Dr. K. Ramalakshmi**, Head of the Department, Computer Science and Engineering (Cyber Security), and **Dr. Vullikanti Vivek**, Deputy Director / ASAC, for their encouragement and cooperation at various levels of the project.

I avail this chance to express my deep sense of gratitude and hearty thanks to the Management of Alliance University, for providing world class infrastructure, congenial atmosphere, and encouragement.

I express my deep sense of gratitude to the teaching and non-teaching staff of our department who stood with me during the project and helped me to make it a successful venture.

I place highest regards to my parents, my friends and well-wishers who helped plenty in making the report of this project.

Amaan Saify (2023BCSE07AED025)

ABSTRACT

Most breaches now start with stolen credentials or an insider, whose actions look legitimate to signature-based tools. User and Entity Behaviour Analytics (UEBA) instead learns how each user and device normally behaves and flags deviations. This project builds a Python UEBA system and takes it from analytics to an operable product.

Four kinds of logs (authentication, system access, network, resource usage) are turned into 44 daily features per user and 19 per device and compared with adaptive personal and department baselines that never learn suspicious days. Robust statistics, an unsupervised ensemble (Isolation Forest, Local Outlier Factor, autoencoder) and 22 MITRE ATT&CK rules are fused into a 0-100 risk score that requires agreement between layers, and every alert explains itself. Around this core, logs arrive by upload, drop folder, an authenticated API or an endpoint agent; a collector service scores new data incrementally - provably identical to a full re-run - and notifies analysts; alerts are grouped into cases; and the console is protected by roles, hashed passwords, scoped tokens and an audit trail.

On a simulated 48-person organisation with 11 planted attacks, all were detected on their first day with precision, recall and F1 of 0.964 (mean F1 0.951 over five organisations), and a live replay through the API reproduced this exactly. On the independent CERT r4.2 benchmark the out-of-the-box system found only 3 of 36 held-out insiders because its behavioural layers ignore small deviations; one generic indicator raised this to 22 of 36 at under one alert a day, while gradual data theft by job seekers remained largely undetected.

Keywords: UEBA, insider threat, anomaly detection, Isolation Forest, autoencoder, risk scoring, explainable alerts, SOC, CERT.

LIST OF SYMBOLS & ABBREVIATIONS

Abbreviation	Meaning
AE	Autoencoder
API	Application Programming Interface
AUC	Area Under the Curve
CERT	Computer Emergency Response Team (CMU insider-threat dataset)
CISO	Chief Information Security Officer
CSV	Comma-Separated Values
DFD	Data Flow Diagram
EDR	Endpoint Detection and Response
FN / FP	False Negative / False Positive
FPR	False Positive Rate
GCN / GNN	Graph Convolutional Network / Graph Neural Network
IF	Isolation Forest
LOF	Local Outlier Factor
LSTM	Long Short-Term Memory
MAD	Median Absolute Deviation
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge
ML	Machine Learning
P95	95th percentile
PBKDF2	Password-Based Key Derivation Function 2
PR-AUC	Area under the Precision-Recall curve
ROC	Receiver Operating Characteristic
SHA-256	Secure Hash Algorithm, 256-bit
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Centre
TLS	Transport Layer Security
TN / TP	True Negative / True Positive
UEBA	User and Entity Behaviour Analytics
VPN	Virtual Private Network
z	Robust z-score (deviation measured in units of typical spread)

CONTENTS

Page No

Title Page.....i

Certificate.....ii

Declaration.....iii

Acknowledgement.....iv

Abstract.....v

List of Symbols & Abbreviations.....vi

1. Introduction1

2. Literature Review2

3. Identification of the Problem / Scope of the Project5

4. Proposed Idea / Solution6

5. Objectives7

6. Process / Methodology8

6.1 Identify the needs of customer and users through Design Thinking Methods.....

6.2 Translate the needs of customer into Project / Product.....

6.3 Specification of the Project / Product.....

6.4 Designing the solution for the customer needs.....

6.5 Process / Flow / Design Diagram.....

7. Requirements14

7.1 Hardware Requirements.....

7.2 Software Requirements.....

7.3 Datasets.....

7.4 Technologies and Tools.....

8. Novelty of the Proposed Idea / Project15

9. Experimentation16

10. Results and Discussion17

11. Conclusion and Future Directions21

References22

Source Code24

Submission Page of the Paper25

Plagiarism Report26

1. INTRODUCTION

Most breaches now start with valid credentials or with an insider, not with malware: credential abuse is the most common initial access vector in the 2025 Verizon DBIR and the human element is involved in about 60% of breaches [1], while insider incidents cost organisations US$17.4 million a year on average [2]. Tools that look for known-bad signatures cannot see these attacks, because every individual log line looks legitimate. What gives them away is behaviour: a salesperson logging in at 3 a.m. from abroad, an engineer opening ninety files instead of fifteen, a laptop uploading gigabytes to a site it has never used.

User and Entity Behaviour Analytics (UEBA) learns what is normal for every user and device and flags deviations [3]. Its practical difficulty is false alarms: analysts already report being flooded by alerts that turn out to be harmless [4]. A useful UEBA system therefore has to be accurate, quiet, explain itself, and fit into how a security team actually works.

This project builds such a system in Python and takes it from analytics to an operable product. It captures authentication, system-access, network and resource-usage logs; profiles users and devices against their own history and their department; detects deviations with statistics, an unsupervised machine-learning ensemble and MITRE ATT&CK rules [5]; and turns the evidence into explained, risk-ranked alerts. Around this core it collects logs continuously, scores new data incrementally, groups alerts into cases for analysts, sends notifications and protects itself with accounts, roles and an audit trail. It is evaluated on a simulated organisation with planted attacks and on the public CMU CERT r4.2 insider-threat benchmark [14].

1.1 Organisation of the Report

Chapter 2 reviews related research; Chapters 3-5 define the problem, the solution and the objectives; Chapter 6 describes the methodology and design, and gives the reason for every feature; Chapters 7 and 8 list the requirements and the novelty; Chapters 9 and 10 present the experiments and results; Chapter 11 concludes.

2. LITERATURE REVIEW

2.1 Paper 1: A User and Entity Behavior Analytics Log Data Set for Anomaly Detection in Cloud Computing [3]

Abstract: IEEE BigData 2022. Publishes CLUE-LDS, real logs of 5,000+ users of a cloud file-sharing service over five years, injects account hijacking by swapping similar users, and proposes an adaptive detector that retrains when behaviour changes.

Accomplishments and limitations: A rare real UEBA data set; 65% true positives at a 4.6% false positive rate. Uses event-type counts from one source, simulated attacks, and no explanation or prioritisation; warns that synthetic data under-states false alarms.

Scope: Supports our adaptive baseline and our caution about synthetic results.

2.2 Paper 2: Multi-homed Abnormal Behavior Detection Algorithm Based on Fuzzy Particle Swarm Cluster in UEBA [6]

Abstract: Scientific Reports 2022. MAD-FPC clusters entity behaviour with fuzzy clustering tuned by particle-swarm optimisation and weights rare behaviour with a TF-IDF-like measure.

Accomplishments and limitations: F1 0.93 on data derived from NSL-KDD. The data is an old intrusion set rather than behaviour logs, the model is static, and anomalies are not explained.

Scope: Supports profiling entities (devices) as well as users, and weighting rare behaviour (our novelty features).

2.3 Paper 3: Cybersecurity Threat Detection Based on a UEBA Framework Using Deep Autoencoders [7]

Abstract: AIMS Mathematics 2025. A deep autoencoder over numeric and Doc2Vec-embedded text fields of a bank's logs; reconstruction residuals explain which fields were anomalous.

Accomplishments and limitations: Handles mixed logs and gives feature-level explanations, but the data had no labels, so no detection metrics; a single model and no risk score.

Scope: The closest explainable UEBA; supports the autoencoder in our ensemble.

2.4 Paper 4: Hunting for Insider Threats Using LSTM-Based Anomaly Detection [8]

Abstract: IEEE TDSC 2023. LADOHD learns the expected order of endpoint events with LSTMs and flags sequences that do not fit, even across days.

Accomplishments and limitations: 97.29% true positives at 0.38% false positives on 38.9 M events from 30 computers; one data source, one organisation, unexplained alerts.

Scope: Shows what sequence models achieve; our results on gradual CERT insiders point to it as future work.

2.5 Paper 5: Dual-Domain Graph Convolutional Network for Insider Threat Detection (DD-GCN) [9]

Abstract: IEEE TIFS 2023. Builds graphs of users' features and relationships and applies graph convolution in two domains.

Accomplishments and limitations: 98.2% / 98.7% accuracy on CERT r4.2 / r6.2, but semi-supervised (needs labelled users), users only, no explanation.

Scope: Supports comparing users with related users; we use simpler, label-free department peer baselines.

2.6 Paper 6: LAN: Learning Adaptive Neighbors for Real-Time Insider Threat Detection [10]

Abstract: IEEE TIFS 2024. Detects malicious activities in real time with graph structure learning over activity sequences and a hybrid self-supervised / supervised loss.

Accomplishments and limitations: At least 9.9% higher AUC than nine baselines on CERT r4.2 / r5.2; uses labels from abnormal activities, users only, no explanation.

Scope: Shows the move to real-time detection; our collector service scores new data within seconds.

2.7 Paper 7: AUTH: An Adversarial Autoencoder Based Unsupervised Insider Threat Detection Scheme [11]

Abstract: IEEE TII 2024. A fully unsupervised adversarial autoencoder (TCN + LSTM encoder) over time and event features from several logs.

Accomplishments and limitations: AUC 0.932 on a public insider data set; users only, no explanation or ranking, no protection against baseline poisoning.

Scope: Supports unsupervised learning and time-of-day features.

2.8 Paper 8: Euler: Detecting Network Lateral Movement via Scalable Temporal Link Prediction [12]

Abstract: ACM TOPS 2023. Treats lateral movement as unlikely new edges in a series of host graphs (GNN + recurrent model).

Accomplishments and limitations: AUC 0.99 on 1.6 B real LANL events, yet precision only about 0.006 - almost every alert false. Hosts only, no explanation.

Scope: Strong evidence that ranking quality is not enough: our design requires corroboration before alerting.

2.9 Paper 9: DEEPCASE: Semi-Supervised Contextual Analysis of Security Events [13]

Abstract: IEEE S&P 2022. Learns the context of security events with attention, clusters similar sequences, and lets analysts label a few per cluster.

Accomplishments and limitations: Cut analysts' manual work by 90.5% on 10.5 M events from 20 organisations; works on existing alerts, needs labels.

Scope: Supports explained alerts, analyst feedback and our grouping of alerts into cases.

2.10 Comparison and Research Gap

Table 2.1: Comparison of the reviewed papers

Paper	Data	Technique	Profiles	Labels	Explained	Key result
[3] 2022	Real cloud logs	Adaptive similarity	Users	No	No	TPR 65%, FPR 4.6%
[6] 2022	NSL-KDD	Fuzzy PSO clusters	Users+entities	No	No	F1 0.93
[7] 2025	Real bank logs	Autoencoder	Users+entities	No	Yes	no labelled metrics
[8] 2023	Real EDR, 30 PCs	LSTM	Endpoints	No	No	TPR 97.3%, FPR 0.38%
[9] 2023	CERT r4.2, r6.2	Graph conv.	Users	Some	No	Acc. 98.2%
[10] 2024	CERT r4.2, r5.2	Graph + sequence	Users	Some	No	+9.9% AUC
[11] 2024	Public insider	Adversarial AE	Users	No	No	AUC 0.932
[12] 2023	LANL, 1.6 B	Temporal GNN	Hosts	No	No	precision 0.006
[13] 2022	20 orgs	Attention	Alerts	Some	Yes	workload -90%
This work	Simulated + CERT	Stats + ML + rules	Users+devices	No	Yes	Ch. 10

Accuracy on public benchmarks is already high [9]-[11], but these systems profile users only, often need labels, and stop at an unexplained score. Entity profiling [6], [12], explanation [7], [13] and adaptive baselines [3] exist separately. None protects its baseline against slow poisoning, none corroborates machine-learning anomalies with rules to cut false alarms - although [12] shows the cost - and none is delivered as a system a security team can operate. The gap addressed here is integration and usability: users and devices profiled together, adaptive but poisoning-resistant baselines, corroborated and explained risk scores, and the collection, workflow and security needed to use them.

2.11 Any Other Sources

MITRE ATT&CK [5] for rule technique IDs; the Verizon DBIR [1] and Cost of Insider Risks report [2] for statistics; the CMU CERT data set [14] as benchmark; studies of SOC analysts [4], [15]-[17] for the personas; and scikit-learn and the original model papers [18]-[21].

3. IDENTIFICATION OF THE PROBLEM / SCOPE OF THE PROJECT

3.1 Problem Identification

Attacks through valid accounts - stolen credentials, account takeover, malicious insiders - look legitimate event by event, so signature tools miss them and damage is found weeks later [2]. Behaviour-based detection can find them, but in practice it:

- **raises too many false alarms** - analysts report most alerts are benign [4]; a high AUC can still mean precision below 1% [12];
- **does not explain itself** - analysts distrust scores they cannot understand [17];
- **profiles users only** - although cryptomining or a pivot laptop shows up on machines;
- **uses static or unprotected baselines** - stale as habits change, or slowly poisoned by an attacker;
- **stops at a score** - with no ranking, case handling, collection or protection of the tool itself.

Problem statement: develop a Python-based UEBA system that captures authentication, system-access, network and resource-usage logs, builds behavioural profiles for users and devices, identifies deviations from normal behaviour, and generates accurate, explained, risk-based alerts that a security team can operate on real logs.

3.2 Scope of the Project

In scope: four log streams from uploads, a drop folder, an authenticated API and an endpoint agent; user and device profiles; three detection layers with risk fusion; cases, suppression and notifications; a secured web console; evaluation on a simulated organisation and on CERT r4.2. Out of scope: per-event streaming, automated response (e.g. disabling an account), parsing native binary formats such as EVTX, and inspecting file or e-mail content - only metadata is analysed, for privacy.

4. PROPOSED IDEA / SOLUTION

A layered pipeline turns raw logs into a short list of explained, risk-ranked alerts, and an operational layer runs it continuously (Figures 6.1 and 6.3):

- **Capture.** Logs arrive by upload, drop folder, ingest API or agent and are mapped to one schema.
- **Features.** Each user-day becomes 44 numbers and each device-day 19 (failed logins, unusual hours, MB to new domains, USB use ...), computed only from earlier days, as a live system would.
- **Profiles.** Every entity has a personal and a department-peer baseline that adapt over time, but suspicious days are never learned.
- **Detection.** Robust statistics, an unsupervised ensemble (Isolation Forest, LOF, autoencoder) and 22 ATT&CK-mapped rules score every day independently.
- **Risk and response.** Scores are fused into a 0-100 risk weighted by asset criticality and accumulated over days; alerts carry plain-English reasons, are grouped into cases and notified.
- **Operation.** A collector service imports new data and scores only what changed with the saved models; the console requires sign-in and records every action.

Three principles run through the design. Corroboration: statistics alone can never raise an alert - the ML ensemble or a rule must agree - which removes most false alarms. Explanation: every alert states what was unusual compared with the entity's own history. Learning with protection: profiles adapt to legitimate change, while high-risk days, rule matches and confirmed threats are never learned.

5. OBJECTIVES

- To capture and normalise authentication, system-access, network and resource-usage logs from files, an API and an endpoint agent into one schema.
- To build adaptive, poisoning-resistant behavioural profiles for both users and devices.
- To detect deviations without attack labels, using complementary statistical, machine-learning and rule layers.
- To produce one explained 0-100 risk score per entity-day and turn alerts into cases analysts can work.
- To operate continuously and securely: incremental scoring identical to a full re-run, notifications, roles, API tokens and an audit trail.
- To evaluate honestly: detect every planted attack with precision above 0.9 and about one alert a day, check robustness, and validate on the independent CERT r4.2 benchmark.

6. PROCESS / METHODOLOGY

6.1 Identify the Needs of Customer and Users through Design Thinking Methods

The personas are composites of four published studies of SOC analysts, as interviews were not possible: analysts are buried in benign alarms [4], judge rules by their noise [15], are 2.5 times more accurate with a structured process [16], and mistrust ML scores they cannot reason about [17].

Table 6.1: Personas (composites of published studies)

Persona	Goal and pain points	Needs from the system
Priya, tier-1 SOC analyst	Triage fast, never miss an attack; flooded by benign alerts with no reasons	Few ranked alerts, a reason for each, context and history in one place
Rahul, SOC manager	Balance workload and coverage; sceptical of black-box AI	Risk ranking, ATT&CK coverage, measurable performance, control of the threshold
Ananya, employee	Not be wrongly accused; worried about intrusive monitoring	Metadata only, fair comparison with her own history, quick clearing of false alerts

Define: help an analyst spot account misuse early without adding to the flood of false alarms, and show why. Ideate: rules alone are blind to new behaviour, one deep model is a black box, so a hybrid was chosen. Prototype and test: the first prototype had precision 0.31; the P95-aware spread, quarantine and the corroboration threshold raised it to 0.96.

6.2 Translate the Needs of Customer into Product

Table 6.2: Needs translated into features - and why each feature exists

Need	Feature	Why it is there
Fewer false alarms	Corroboration threshold 45; P95-aware spread	One unusual number must not raise an alert; personal monthly peaks are normal
Know why	Plain-English reasons, layer scores, ATT&CK IDs	Analysts act on alerts they understand [4], [17]
Know what to open first	0-100 risk, criticality, severity	Queues are ranked, not flat
Catch slow attacks	Accumulated risk, 3-day half-life	Small daily signals add up
Not drown in repeats	Cases; suppression with expiry	One person = one work item; known patterns go quiet
Learn from analysts	Verdicts feed the baselines	Cleared days are learned, threats never
Use real logs	Upload with column mapping; drop folder; API; agent	Logs come in many formats and continuously
Timely alerts	Collector service; incremental engine; notifications	Nobody watches a dashboard all day
Trust in the tool	Roles, hashed passwords, tokens, audit	A tool holding security logs must be secure
Privacy, fairness	Metadata only; own and peer baselines	Compare people with themselves, not a global norm

6.3 Specification of the Product

Table 6.3: Main functional requirements

ID	Requirement
FR1	Accept the four log streams as CSV, JSON, ZIP, Linux auth.log / syslog text or Windows event exports (CSV, .evtx); split mixed files; map columns by name and content; normalise values.
FR2	Collect continuously from a drop folder, a token-authenticated HTTP API and an endpoint agent.
FR3	Compute 44 user and 19 device features per day; learn personal and peer baselines over a rolling 45 days.
FR4	Score each day with statistics, the ML ensemble and 22 rules; fuse into a 0-100 risk with criticality and decay.
FR5	After the learning period, score new data incrementally with saved models, identical to a full re-run.
FR6	Raise explained alerts; group them into cases with status, assignee and notes; allow audited suppression.
FR7	Notify analysts of new or escalated alerts by webhook or e-mail, without duplicates.
FR8	Require sign-in with viewer / analyst / administrator roles; scope API tokens; audit every action.
FR9	Measure precision, recall, F1, false positive rate, ROC-AUC and PR-AUC against labelled incidents.

Table 6.4: Key parameters

Parameter	Value	Purpose
Learning period	30 days (configurable)	Learn normal behaviour before alerting
Adaptive window	last 45 accepted days	Keep profiles current
Layer weights	stats 0.28, peer 0.10, ML 0.30	Fusion into daily risk
Alert threshold	45 (per workspace)	Statistics alone (max 43.8) cannot alert
Quarantine	risk >= 60 or a rule hit	Protect baselines from poisoning
Criticality	admin/server 1.3, finance 1.2, HR 1.1	Weight by impact of compromise
Risk half-life	3 days	Accumulate slow behaviour
Case gap	7 days	Alerts closer than this form one case

6.4 Designing the Solution for the Customer Needs

6.4.1 Features and profiles

Each user-day is summarised into 44 features in four groups - authentication (failed logins, failures in 10 minutes followed by success, new hosts and countries, logins at hours rare for this user, impossible travel), system access (sensitive and restricted files, privilege escalation, new resources, account creation, log clearing, USB use), network (MB uploaded, MB to new domains and to cloud storage, mining-pool connections, job sites) and resource use (CPU, memory, disk) - and each device-day into 19. Why: daily aggregates are cheap, private and explainable, and novelty features are computed only from earlier days, so the system behaves the same in a test as in live use.

For each entity and feature the profile stores centre = median and spread = max(1.4826 x MAD, (P95 - median) / 1.645, floor); deviation is z = (today - centre) / spread. Why robust statistics: behaviour is spiky, and the P95 term stops a person's own occasional peaks from being flagged. The same profile is built for the department (peer baseline). During detection each profile is rebuilt from the last 45 accepted days; days with risk 60 or more, a rule hit or a confirmed-threat verdict are quarantined. Why: profiles must follow legitimate change without letting an attacker teach the model that the attack is normal.

6.4.2 Detection layers and risk

- **Statistics:** the three largest positive deviations above z = 3, for the personal and the peer baseline.
- **ML ensemble:** Isolation Forest [19], LOF [20] and an autoencoder [21] (scikit-learn [18]) trained on normal days only and calibrated so that 1 = the 99th percentile of normal. Why three: they fail differently; why unsupervised: real organisations have almost no labelled attacks.
- **Rules:** 22 rules mapped to ATT&CK, e.g. brute force (T1110), impossible travel (T1078), privilege escalation by a non-admin (T1068), upload to a new domain (T1048), mining pools (T1496), log clearing (T1070), removable media used by an account with no such history (T1052.001, added after the CERT evaluation, Section 10.3) and a successful login from an address that just failed against several accounts (password spraying, T1110.003). Privilege escalation is judged against the account's own history, so routine administrator sudo is not flagged even without a staff list.
- **Fusion:** A = 1 - (1 - 0.28 stat)(1 - 0.10 peer)(1 - 0.30 ml)(1 - rules); daily risk = 100 x min(1, A x criticality); entity risk(t) = entity risk(t-1) x 0.5^(days/3) + max(0, daily risk - 15). An alert is raised at daily risk 45 (critical at 80, high at 60) or when accumulated risk reaches 60. Why 45: the statistical layers alone reach at most 43.8, so an alert always needs the ML ensemble or a rule to agree.

6.4.3 Continuous operation

Each monitored environment is a workspace with its own logs, models, alerts and verdicts, moving from learning to monitoring once its learning period is covered. Logs arrive by upload (automatic column mapping by column name and content, such as TimeCreated to timestamp, with a preview), a drop folder, a token-authenticated ingest API or an endpoint agent that keeps data while the server is unreachable. Linux auth.log / syslog text and Windows Security events (Event Viewer CSV, PowerShell export, .evtx) are read directly, and a file mixing several event types is split by its event-type column. Why four routes: real logs come from exports, scripts, shippers and machines, and all of them end in the same import step.

The engine trains the baselines and ML models once and saves them with a fingerprint of the settings; later analyses reuse them and score only the days from the earliest changed day onwards. Because every feature is causal, the result is identical to a full re-run - checked by an automated test at a day boundary and mid-day. Why: re-training on all history at every update does not scale and would change past scores. A collector service runs every 10 seconds (import, incremental analysis, notification) under a file lock, and an analyst verdict marks the following days for re-scoring so that cleared behaviour is learned.

6.4.4 Analyst workflow and security

- **Cases:** alerts on one entity within 7 days form a case with status, assignee and a notes timeline; closing it records a verdict for every day. Why: an analyst should handle a person once, not 30 times.
- **Suppression:** a known-benign pattern can be silenced for one entity and alert type until an expiry date, with a mandatory reason; suppressed alerts are kept and audited. Why: on CERT one user produced 91 alerts for one habit.
- **Notifications:** webhook (Slack, Teams) or e-mail for new or escalated alerts, never twice, quiet for imported history. Why: detection is useless if nobody sees it in time.
- **Security:** viewer / analyst / administrator roles; PBKDF2 password hashes with lock-out after five failures; 256-bit API tokens stored only as hashes and scoped to a workspace; an audit trail of every login, verdict and setting; services bound to the local machine by default. Why: the console holds the organisation's security logs.

6.5 Process / Flow / Design Diagram

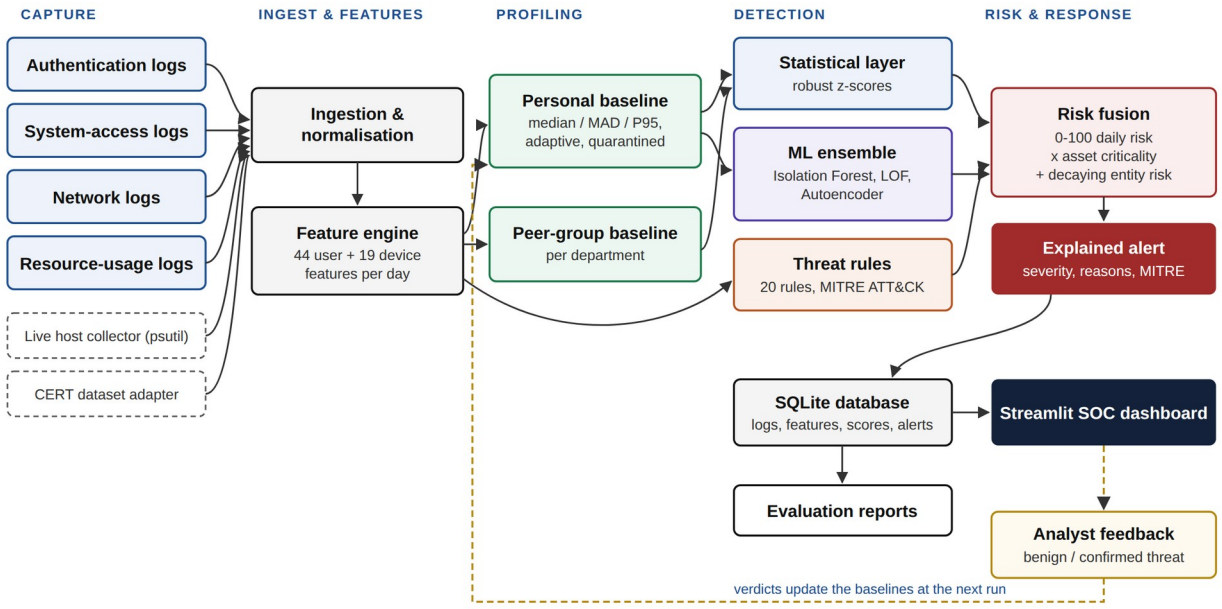


Figure 6.1: Analytics architecture

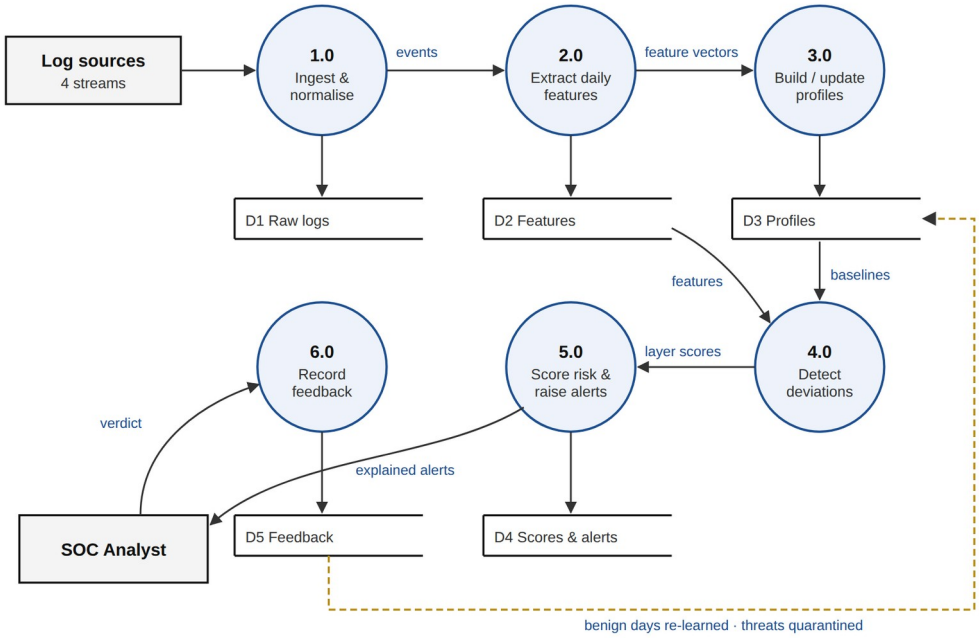


Figure 6.2: Data flow diagram, level 1

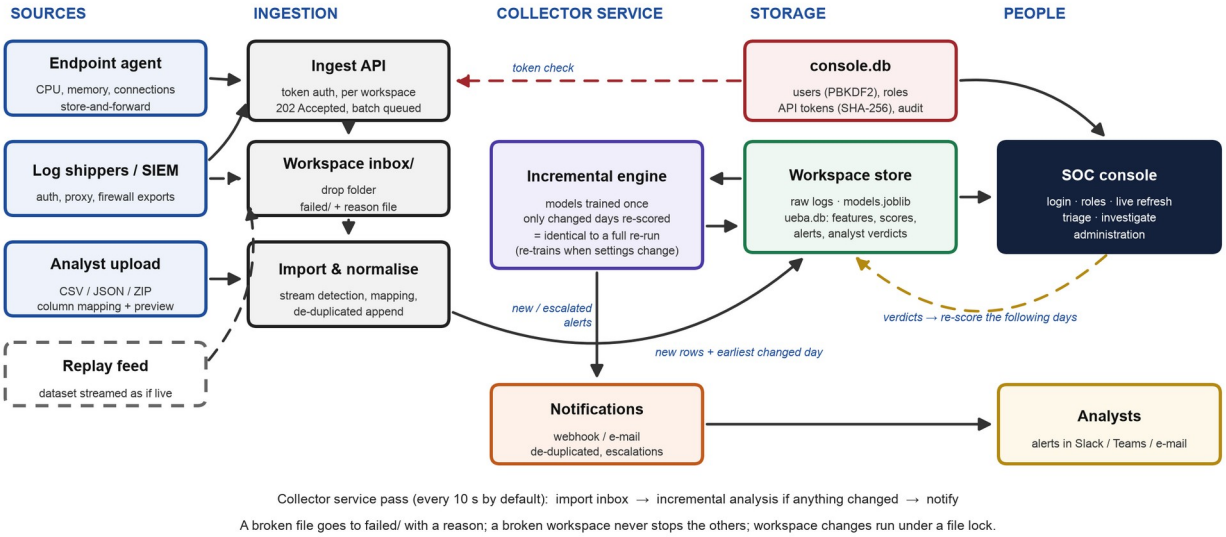
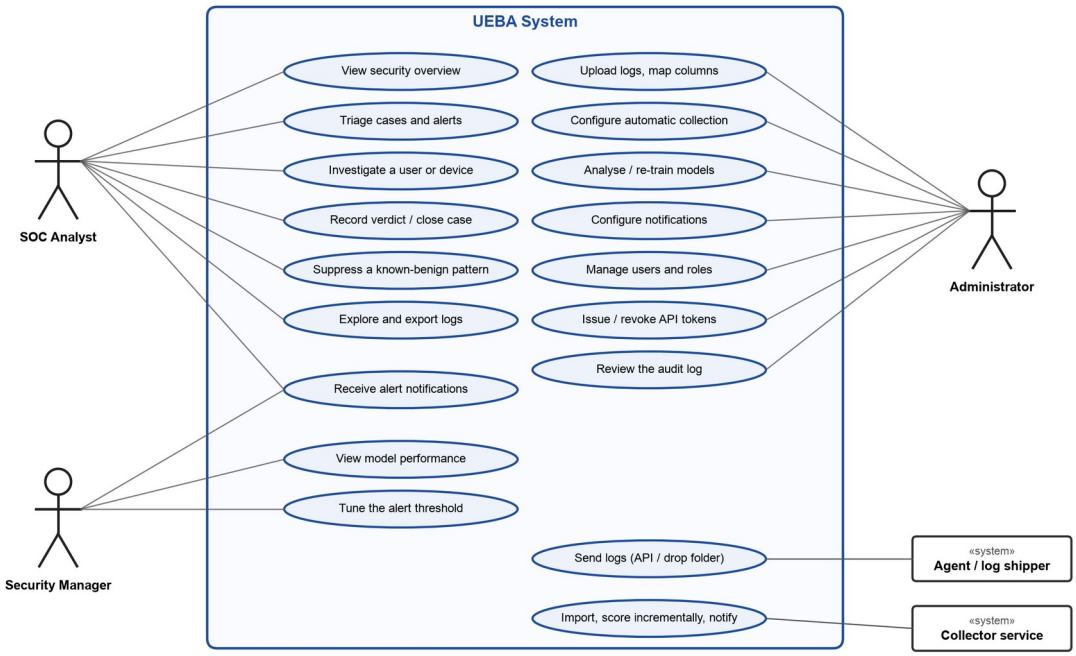


Figure 6.3: Operational architecture - how logs arrive, are scored continuously and reach analysts



Every person signs in. Viewers can only read; analysts can also triage; administrators can also manage data, settings, users and tokens.

Figure 6.4: Use case diagram

7. REQUIREMENTS

7.1 Hardware Requirements

Any 64-bit dual-core computer with 4 GB of RAM and 1 GB of disk runs the simulated organisation; the full CERT benchmark needs 16 GB of RAM (6.4 GB peak) and 25 GB of disk. No GPU is needed. Development used a 16 GB Apple-silicon laptop.

7.2 Software Requirements

Table 7.1: Software

Software	Version	Purpose
Python	3.10 or later (3.14 used)	Language
pandas, NumPy	2.0+, 1.24+	Log processing and features
scikit-learn	1.3+	Isolation Forest, LOF, autoencoder
Streamlit, Plotly	1.50+, 5.18+	Analyst console and charts
SQLite	bundled	Logs, scores, alerts, cases, users, audit
Python standard library	bundled	Ingest API, password hashing, notifications, locking
psutil, Matplotlib, pytest	5.9+, 3.7+, 7.4+	Agent telemetry, report figures, 32 tests

7.3 Datasets

- **Simulated organisation (primary):** 48 employees and 12 servers over 60 days, 205,483 events, 11 planted attacks (28 attack entity-days) and 4 harmless but unusual events. Why: real logs are confidential and never labelled.
- **CMU CERT r4.2 [14]:** 1,000 simulated employees over 17 months with 70 insiders in three scenarios; the full 16 GB data set was converted into 12.8 million events. Why: an independent benchmark the author did not design.
- **Live data:** the endpoint agent and the replay of a data set as a live feed.

7.4 Technologies and Tools

Python, pandas, NumPy, scikit-learn, Streamlit, Plotly, SQLite and the standard library (http.server, hashlib, smtplib, fcntl); MITRE ATT&CK for rule mapping; Git for version control; pytest for testing.

8. NOVELTY OF THE PROPOSED IDEA / PROJECT

The individual techniques are established; the novelty is how they are combined and delivered (Table 8.1).

Table 8.1: Comparison with the reviewed work

Capability	Reviewed work	This project
Users and devices profiled together	users [9]-[11] or hosts [12]	44 user + 19 device features
Protection against baseline poisoning	not addressed	quarantine of risky days and confirmed threats
No attack labels needed	labels in [9], [10], [13]	unsupervised + rules
Corroborated, explained risk score	single model, score only	threshold needs agreement; plain-English reasons
Exact incremental scoring	offline batch evaluation	saved models; identical to a full re-run (tested)
Operable and secured	not addressed	collection, cases, notifications, roles, tokens, audit

- **Poisoning-resistant adaptive baselines:** profiles follow legitimate change, but risky days, rule hits and confirmed threats are never learned, while cleared days are.
- **Corroboration by design:** the threshold is derived from the score formula so that one statistical deviation can never alert - the main reason precision rose from 0.31 to 0.96.
- **Exact incremental scoring:** because every feature is causal, scoring only new days is provably the same as re-analysing everything, and an automated test checks it.
- **Honest validation:** an independent benchmark with a held-out split and a documented negative result.

9. EXPERIMENTATION

9.1 Experimental Setup

The simulated organisation has 48 employees in six departments, each with a laptop and personal habits, plus 12 servers (108 entities). Days 1-5 are warm-up, days 6-30 the attack-free baseline, days 31-60 the detection window, which contains the 11 attacks of Table 9.1 and four harmless oddities (working until 1 a.m., a business trip, a 20 GB backup, a new project) that create realistic false-alarm pressure.

Table 9.1: Planted attacks

ID	Scenario	What appears in the logs	ATT&CK
INC-01	Brute force + takeover	45 failed VPN logins from Tor, then success and upload	T1110
INC-02 / 10	Impossible travel	Bengaluru, then Amsterdam / Lagos 70 minutes later	T1078
INC-03	Off-hours access	02:20 login, payroll read, WeTransfer upload	T1078
INC-04 / 05	Exfiltration / slow insider	3.3 GB to mega.nz; five days of growing Dropbox uploads	T1567
INC-06	Privilege escalation	Sales user runs admin commands on the domain controller	T1068
INC-07	Lateral movement	SSH/RDP into 12 new hosts in 50 minutes	T1021
INC-08	Cryptomining	Laptop CPU ~96% for three days, mining-pool traffic	T1496
INC-09	Password spray	30 failed logins on an admin account	T1110
INC-11	Admin account abuse	Accounts created, security log cleared	T1136, T1070

9.2 Evaluation Protocol and Metrics

Every entity-day of the detection window is a test case, positive if labelled as an attack. Labels are used only for scoring, never for training. Metrics: precision, recall, F1-score, false positive rate, ROC-AUC and PR-AUC (which matters because attacks are about 1% of days), incidents detected, and - for CERT - insiders found within a fixed daily analyst review budget.

9.3 Experiments Conducted

- **Main run, robustness and ablation:** seed 42; five differently generated organisations; layers switched off.
- **CERT r4.2:** learning January-May 2010 (before any attack); users split at random into a development half and a held-out test half; the out-of-the-box result frozen first; one change (R16) chosen on the development half only; a 7-day trend layer evaluated the same way.
- **Operation:** incremental update compared with a full re-run; the organisation replayed through the ingest API as a live feed.
- **Tests:** 32 automated tests of detection, ingestion, the incremental engine, the API, security, cases and notifications.

10. RESULTS AND DISCUSSION

10.1 Simulated Organisation

Table 10.1: Detection results (detection window, seed 42)

Metric	Value
Incidents detected	11 of 11, all on the day they started
Precision / recall / F1-score	0.964 / 0.964 / 0.964 (TP 27, FP 1, FN 1)
False positive rate / alerts per day	0.04% / about 0.9 for 108 entities
Fused ROC-AUC / PR-AUC	1.000 / 0.999
Five organisations (mean)	precision 0.938, recall 0.964, F1 0.951; 55 of 55 incidents
Behavioural layers only (no rules)	F1 0.857, PR-AUC 0.951
Rules only / ML only / statistics only	F1 0.982 / 0.789 / 0.451

The only false positive was the planted new-project oddity (risk 49), and the only false negative the pivot laptop of the lateral movement (43.6, just below the threshold). The ablation needs careful reading: rules look best only because they were written for these attack types; without any rule the behavioural layers still reach F1 0.857 with no prior knowledge of the attacks. Statistics alone flag nearly everything unusual (precision 0.30), which is why the threshold requires corroboration.

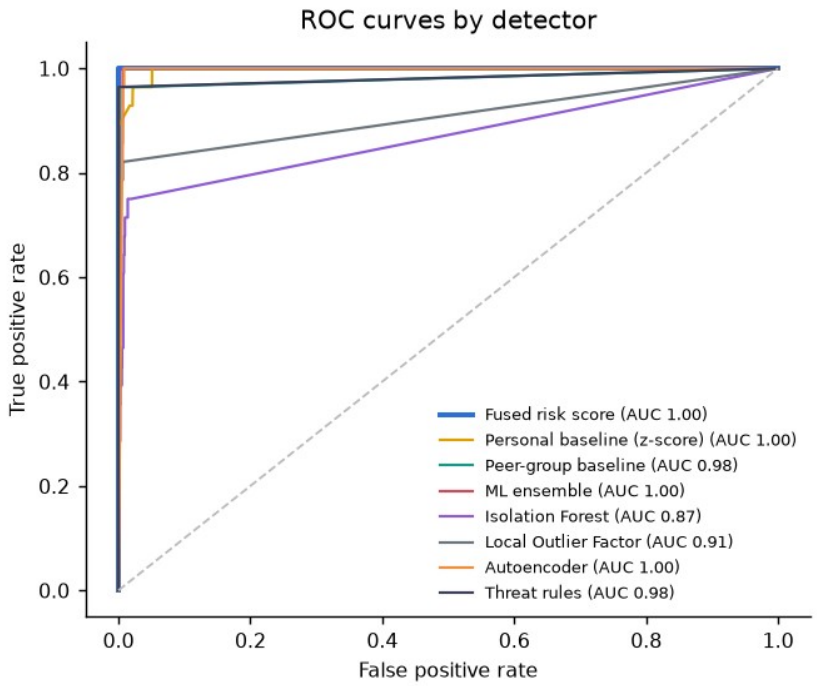


Figure 10.1: ROC curves of each detection layer (simulated organisation)

10.2 Continuous Operation

Cutting the logs at a day boundary and in the middle of a day, then updating incrementally, gave scores, explanations and alerts identical to a full re-run. Replaying the organisation through the ingest API (30 days of history, then 30 live days in 8-hour slices) took 2 full trainings and 29 incremental updates of 1.6-3.2 seconds, with no rejected file or error, and reproduced the batch result exactly: 11 of 11 incidents,

each in the service pass after its data arrived, and F1 0.964. A full analysis of the organisation takes about 5 seconds; CERT, with 12.8 million events, about 8 minutes.

10.3 CERT r4.2 Benchmark

Table 10.2: CERT r4.2, held-out half of the users (36 labelled accounts)

Measure	Out-of-the-box	With R16
Insiders alerted on a malicious day	3 / 36	22 / 36
Scenario 1: new USB user leaks data	1 / 18	18 / 18
Scenario 2: job seeker steals data	1 / 14	1 / 14
Scenario 3: administrator, keylogger	1 / 4	3 / 4
Alerts per day (about 500 users)	0.66	0.84
Day-level precision / recall	0.017 / 0.008	0.146 / 0.091
Fused ROC-AUC	0.699	0.733
Insiders among the 3 riskiest users per day	12 / 36	28 / 36

Out of the box the system generalised poorly: 3 of 36 held-out insiders. The cause is calibration. The behavioural layers were tuned on loud simulated attacks and deliberately ignore deviations below three robust standard deviations and ML scores below the 99th percentile - which is what keeps false alarms to about one a day - but CERT insiders are quiet (a first USB connection, a few job-site visits), so the statistical and ML layers rank CERT days barely better than chance (ROC-AUC about 0.54). The answer key labels 72 accounts: the 70 insiders and two supervisors whose accounts scenario-3 insiders misused.

One generic insider indicator was then added: removable-media use by an account that never used removable media in its learning period (R16, T1052.001, weight 0.45 like comparable rules). It never fires on the simulated organisation and was chosen on the development half (4 to 17 of 36). On the untouched test half it raised detection to 22 of 36 - every scenario-1 insider - at 0.84 alerts a day, and an analyst reviewing the three riskiest users each day (0.3%) would see 28 of 36. Job seekers who steal data gradually (scenario 2) remain largely undetected because they already used USB drives and only used them more.

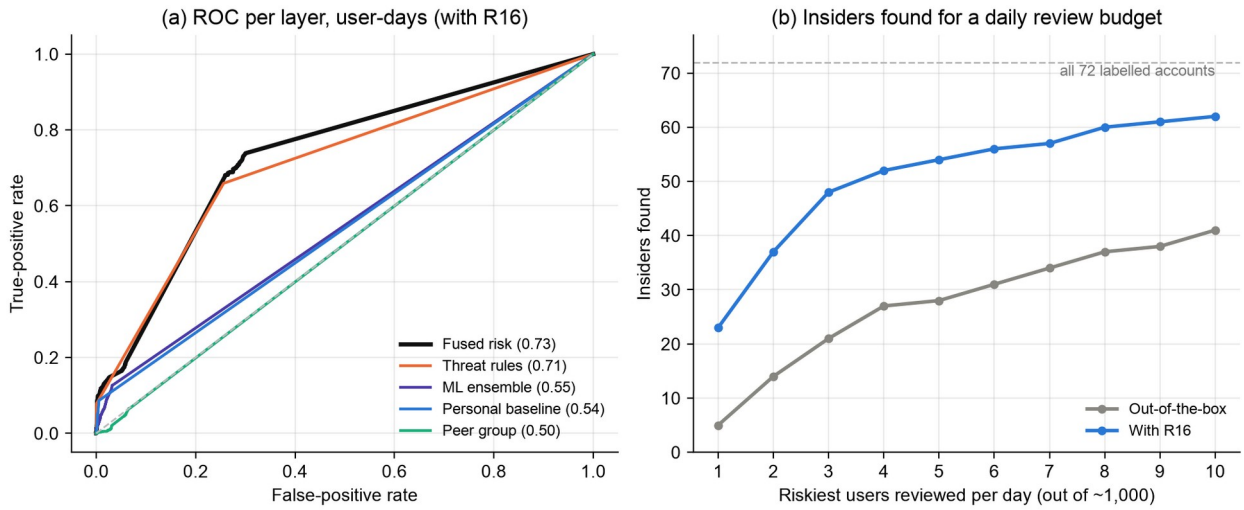


Figure 10.2: CERT r4.2 - (a) ROC curve of each layer, (b) insiders found for a daily review budget (all 72 labelled accounts)

A 7-day trend layer was tried for these gradual insiders. It raised ROC-AUC by about 0.1, but at a weight that keeps about one alert a day it caught no additional insider, at a weight that caught half of the scenario-2 insiders it multiplied alerts by eleven at 3-4% precision, and every weight lowered the number of insiders in the analysts' daily top five. It was therefore not adopted; gradual insiders need sequence-aware models [8], [10].

10.4 Analyst Console

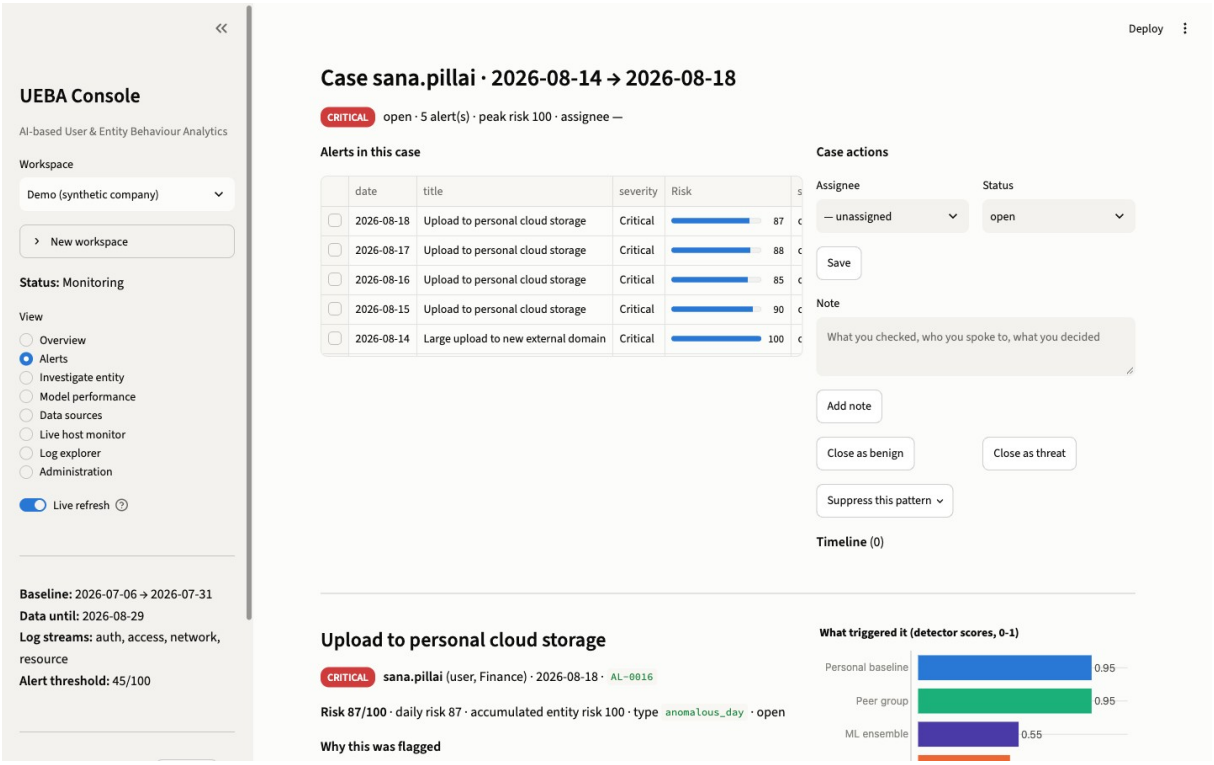


Figure 10.3: A case - five daily alerts of the slow-insider attack grouped into one work item with actions, notes and explained alert detail

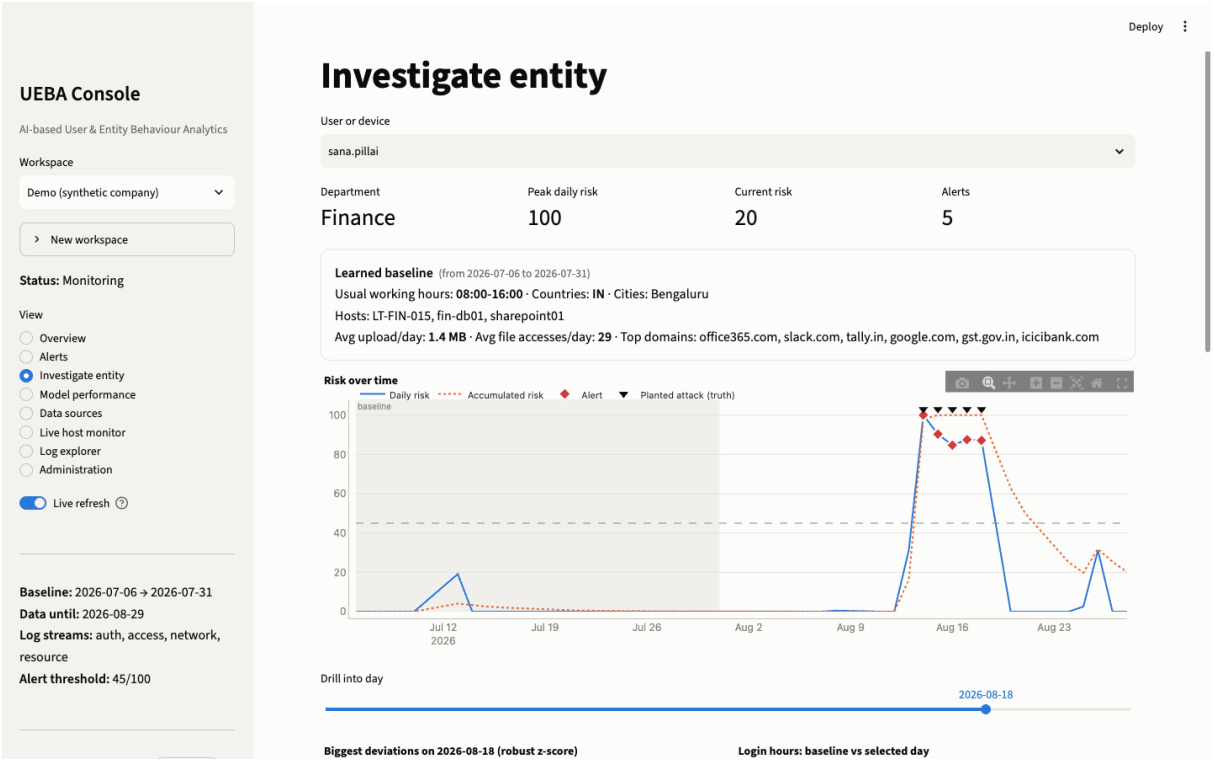


Figure 10.4: Investigating an entity - learned baseline, risk over time with alerts and the labelled attack days

10.5 Discussion

On the simulated organisation the system meets its objectives: every attack on its first day, about one alert a day, and a reason for each. CERT shows that this should be read as an upper bound - the organisation was designed together with the detector, and on independent data the conservative calibration that keeps false alarms low also hides quiet insiders. One generic indicator recovered most of them at under one alert a day, and the analyst-budget view shows the ranking is useful even where alerts are not raised. The threshold, weights and calibration should therefore be tuned per organisation, which the console supports, and gradual insiders remain the main open problem.

11. CONCLUSION AND FUTURE DIRECTIONS

11.1 Conclusion

The project delivers a Python UEBA system that meets the problem statement and runs as a product. It captures four log streams from uploads, a drop folder, an authenticated API and an endpoint agent; profiles users and devices against adaptive, poisoning-resistant personal and peer baselines; detects deviations with robust statistics, an unsupervised ensemble and 22 ATT&CK rules; and turns the evidence into explained, risk-ranked alerts grouped into cases, notified to analysts and handled in a secured console. Incremental scoring is identical to a full re-run.

On the simulated organisation it detected all 11 attacks on their first day with precision, recall and F1 of 0.964 (mean F1 0.951 over five organisations), and the behavioural layers alone reached 0.857. On the independent CERT r4.2 benchmark the out-of-the-box system found only 3 of 36 held-out insiders; with one generic indicator it found 22 of 36 at under one alert a day, while gradual job-seeker theft remained largely undetected - an honest result that defines the next step.

11.2 Future Directions

- **Quiet insiders:** behavioural scores that let many small deviations add up, calibrated per organisation, and sequence models [8], [10] for gradual behaviour.
- **More validation:** CERT r5.2 / r6.2 and the real CLUE-LDS logs [3].
- **Native formats and scale:** EVTX, Sysmon and syslog parsers, PostgreSQL storage, TLS in front of the console.
- **Automated response:** SOAR integration to disable an account or isolate a host once a threat is confirmed.

REFERENCES

[1] Verizon Business, "2025 Data Breach Investigations Report," Verizon, 2025. [Online]. Available: https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf

[2] Ponemon Institute and DTEX Systems, "2025 Cost of Insider Risks Global Report," Feb. 2025. [Online]. Available: https://www.globenewswire.com/news-release/2025/02/25/3032261/0/en/Ponemon-Cybersecurity-Report-Insider-Risk-Management-Enabling-Early-Breach-Detection-and-Mitigation.html

[3] M. Landauer, F. Skopik, G. Höld, and M. Wurzenberger, "A user and entity behavior analytics log data set for anomaly detection in cloud computing," in Proc. IEEE Int. Conf. Big Data (Big Data), Osaka, Japan, Dec. 2022, pp. 4285-4294, doi: 10.1109/BigData55660.2022.10020672.

[4] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in Proc. 31st USENIX Security Symp., Boston, MA, USA, Aug. 2022, pp. 2783-2800.

[5] The MITRE Corporation, "MITRE ATT&CK," 2026. [Online]. Available: https://attack.mitre.org/

[6] J. Cui, G. Zhang, Z. Chen, and N. Yu, "Multi-homed abnormal behavior detection algorithm based on fuzzy particle swarm cluster in user and entity behavior analytics," Sci. Rep., vol. 12, Art. no. 22349, Dec. 2022, doi: 10.1038/s41598-022-26142-w.

[7] J. Fuentes, I. Ortega-Fernández, N. M. Villanueva, and M. Sestelo, "Cybersecurity threat detection based on a UEBA framework using deep autoencoders," AIMS Math., vol. 10, no. 10, pp. 23496-23517, 2025, doi: 10.3934/math.20251043.

[8] M. Villarreal-Vasquez, G. Modelo-Howard, S. Dube, and B. Bhargava, "Hunting for insider threats using LSTM-based anomaly detection," IEEE Trans. Dependable Secure Comput., vol. 20, no. 1, pp. 451-462, 2023, doi: 10.1109/TDSC.2021.3135639.

[9] X. Li, X. Li, J. Jia, L. Li, J. Yuan, Y. Gao, and S. Yu, "A high accuracy and adaptive anomaly detection model with dual-domain graph convolutional network for insider threat detection," IEEE Trans. Inf. Forensics Security, vol. 18, pp. 1638-1652, 2023, doi: 10.1109/TIFS.2023.3245413.

[10] X. Cai, Y. Wang, S. Xu, H. Li, Y. Zhang, Z. Liu, and X. Yuan, "LAN: Learning adaptive neighbors for real-time insider threat detection," IEEE Trans. Inf. Forensics Security, vol. 19, pp. 10157-10172, 2024, doi: 10.1109/TIFS.2024.3488527.

[11] X. Zhu, J. Dong, J. Qi, Z. Zhou, Z. Dong, Y. Sun, and M. Wang, "AUTH: An adversarial autoencoder based unsupervised insider threat detection scheme for multisource logs," IEEE

Trans. Ind. Informat., vol. 20, no. 9, pp. 10954-10965, Sep. 2024, doi: 10.1109/TII.2024.3393491.

[12] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," ACM Trans. Privacy Security, vol. 26, no. 3, pp. 1-36, 2023, doi: 10.1145/3588771.

[13] T. van Ede et al., "DEEPCASE: Semi-supervised contextual analysis of security events," in Proc. IEEE Symp. Security Privacy (SP), San Francisco, CA, USA, May 2022, pp. 522-539, doi: 10.1109/SP46214.2022.9833671.

[14] B. Lindauer, "Insider Threat Test Dataset," Carnegie Mellon Univ., Software Engineering Institute, 2020, doi: 10.1184/R1/12841247.v1.

[15] M. Vermeer, N. Kadenko, M. van Eeten, C. Gañán, and S. Parkin, "Alert alchemy: SOC workflows and decisions in the management of NIDS rules," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Copenhagen, Denmark, Nov. 2023, pp. 2770-2784, doi: 10.1145/3576915.3616581.

[16] L. Kersten, T. Mulders, E. Zambon, C. Snijders, and L. Allodi, "'Give me structure': Synthesis and evaluation of a (network) threat analysis process supporting tier 1 investigations in a security operation center," in Proc. 19th Symp. Usable Privacy Security (SOUPS), Anaheim, CA, USA, Aug. 2023, pp. 97-111.

[17] S. Oesch et al., "An assessment of the usability of machine learning based tools for the security operations center," in Proc. IEEE Int. Conf. iThings/GreenCom/CPSCom/SmartData/Cybermatics, Rhodes, Greece, Nov. 2020, pp. 634-641, doi: 10.1109/iThings-GreenCom-CPSCom-SmartData-Cybermatics50389.2020.00111.

[18] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[19] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), Pisa, Italy, 2008, pp. 413-422.

[20] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in Proc. ACM SIGMOD Int. Conf. Management of Data, Dallas, TX, USA, 2000, pp. 93-104.

[21] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," Science, vol. 313, no. 5786, pp. 504-507, 2006.

SOURCE CODE

The complete source code - about 6,000 lines of Python in 22 modules, the analyst console and 32 automated tests - is in the project folder and its Git repository. Three key parts of the detection logic are reproduced below.

Risk fusion and accumulated risk (ueba/risk.py)

```
def daily_risk(df: pd.DataFrame, cfg: Config = CONFIG) -> np.ndarray:
    a = 1 - ((1 - cfg.w_stat * df["stat"]) * (1 - cfg.w_peer * df["peer"]) *
             (1 - cfg.w_ml * df["ml"]) * (1 - df["rule"]))
    crit = df["department"].map(cfg.criticality).fillna(1.0)
    crit = np.where(df["is_admin"].astype(bool), np.maximum(crit, cfg.criticality.get("IT-Admin",
1.3)), crit)
    return np.round(100 * np.clip(a * crit, 0, 1), 1)


def entity_risk(df: pd.DataFrame, cfg: Config = CONFIG) -> np.ndarray:
    """Decayed accumulation of daily risk above the noise floor (per entity, chronological)."""
    out = np.zeros(len(df))
    decay_per_day = 0.5 ** (1 / cfg.risk_half_life_days)
    order = df.sort_values(["entity", "date"]).index
    prev_ent, prev_date, r = None, None, 0.0
    dates = pd.to_datetime(df["date"])
    for idx in order:
        ent, d = df.at[idx, "entity"], dates.at[idx]
        if ent != prev_ent:
            r = 0.0
        else:
            r *= decay_per_day ** max(1, (d - prev_date).days)
        r = min(100.0, r + max(0.0, df.at[idx, "daily_risk"] - NOISE_FLOOR))
        out[df.index.get_loc(idx)] = r
        prev_ent, prev_date = ent, d
    return np.round(out, 1)
```

Robust baseline spread (ueba/pipeline.py)

```
def _robust(hist: np.ndarray, floor: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Median and robust scale of an entity's history (same formula as profiles._scales)."""
    med = np.median(hist, axis=0)
    mad = np.median(np.abs(hist - med), axis=0) * 1.4826
    tail = (np.quantile(hist, 0.95, axis=0) - med) / 1.645
    return med, np.maximum(np.maximum(mad, tail), floor)
```

Quarantine, analyst feedback and incremental reuse (excerpt of score_entities, ueba/pipeline.py)

```
    def accept(idx):
        # quarantine high-risk days and anything matching a threat signature; medium,
        # purely behavioural deviations are absorbed so the profile adapts to new habits
        accepted[idx] = ((S.loc[idx, "daily_risk"].to_numpy() < cfg.quarantine_risk)
                         & (S.loc[idx, "rule"].to_numpy() < 0.2))
        for i in idx:
            key = (F.at[i, "entity"], F.at[i, "date"])
            if benign and key in benign:
                accepted[i] = True
            elif threat and key in threat:
                accepted[i] = False
    ...
    kept = np.zeros(len(F), dtype=bool)          # rows whose previous scores are reused
    if models and prev is not None and rescore_from:
        P = prev.drop_duplicates(["entity", "date"]).set_index(["entity", "date"])
        keys = pd.MultiIndex.from_arrays([F["entity"], F["date"]])
        known = keys.isin(P.index)
        gap = (~train_mask) & (dates < rescore_from) & ~known      # back-filled days -> rescore
from there
        ...
```

SUBMISSION PAGE OF THE PAPER

PLAGIARISM REPORT